EXPERIMENT 10

Aim

To create a Dockerfile for a Python Flask web application, build a Docker image, run a container, and implement multiple routes including dynamic routing, forms, calculator functionality, and a JSON API.

Theory

What is Docker?

Docker is a containerization platform that allows developers to package applications along with their dependencies into containers. Containers ensure that the application runs consistently across different environments.

What is a Dockerfile?

A Dockerfile is a text file containing instructions that Docker uses to build an image. Each instruction creates a new layer in the image.

Basic syntax:

DIRECTIVE argument

Common directives:

- FROM
- WORKDIR
- RUN
- COPY
- ENV
- CMD

What is Flask?

Flask is a lightweight Python web framework used to build web applications and APIs.

Implementation

Step 1: Create Project Directory

Open Command Prompt and create a new folder:

```
mkdir flask-docker-app
cd flask-docker-app
```

Step 2: Create Flask Application (app.py)

Create a file named:

app.py

Add the following code:

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/')
def home():
  return """
  <h1>🚀 Welcome to My Advanced Docker Flask App</h1>
  <ul>
     <li><a href="/hello/Student">Dynamic Hello</a></li>
     <li><a href="/calculator">Calculator</a></li>
     <li><a href="/form">Form</a></li>
     <li><a href="/api/data">JSON API</a></li>
  </ul>
  """
```

```
@app.route('/hello/<name>')
def hello(name):
    return f"<h2>Hello, {name}!</h2>"

@app.route('/calculator', methods=['GET', 'POST'])
def calculator():
    if request.method == 'POST':
        num1 = float(request.form['num1'])
        num2 = float(request.form['num2'])
        result = num1 + num2
        return f"<h2>Result: {result}</h2>"

    return """
    <form method="POST">
        Number 1: <input type="number" name="num1"><br><br>
        Number 2: <input type="number" name="num2"><br><br>
        <input type="submit" value="Add">
    </form>
    """

@app.route('/form', methods=['GET', 'POST'])
def form():
    if request.method == 'POST':
        username = request.form['username']
        return f"<h2>Welcome, {username}!</h2>"

    return """
    <form method="POST">
        Enter Name: <input type="text" name="username">
        <input type="submit" value="Submit">
    </form>
    """

@app.route('/api/data')
def api_data():
    return jsonify({
        "message": "Dockerized Flask App",
        "status": "success"
    })

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

```
from flask import Flask, request, jsonify

app = Flask(__name__)

# ---------------- HOME PAGE ----------------
@app.route('/')
def home():
    return """
    <h1>🚀 Welcome to My Advanced Docker Flask App</h1>
    <p>Select a feature below:</p>
    <ul>
        <li><a href="/hello/Geet">Dynamic Hello</a></li>
        <li><a href="/add?num1=5&num2=10">Quick Add (5 + 10)</a></li>
        <li><a href="/calculator">Web Calculator</a></li>
        <li><a href="/form">Simple Form</a></li>
        <li><a href="/api/data">JSON API</a></li>
    </ul>
    """

# ---------------- DYNAMIC ROUTE ----------------
@app.route('/hello/<name>')
def hello(name):
    return f"""
    <h2>👋 Hello, {name}!</h2>
    <a href="/">← Back Home</a>
    """

# ---------------- ADD USING QUERY PARAMS ----------------
@app.route('/add')
def add():
    try:
        num1 = float(request.args.get('num1', 0))
        num2 = float(request.args.get('num2', 0))
```

Step 3: Create Dockerfile

Create a file named:

Dockerfile

Add:

FROM python:3.11-slim

WORKDIR /app

RUN pip install flask==2.3

COPY . /app

ENV FLASK_APP=app.py

CMD ["flask", "run", "--host=0.0.0.0", "--port=5000"]

Step 4: Create .dockerignore File

Create file:

.dockerignore

Add:

Dockerfile
__pycache__

Step 5: Build Docker Image

Run:

docker build -t sample-flask-app:v1 .

```
C:\Users\Lab805_19\Desktop\hi\flask-docker-app>docker build -t sample-flask-app:v3 .
[+] Building 1.3s (9/9) FINISHED                                        docker:default
 => [internal] load build definition from Dockerfile                                0.0s
 => => transferring dockerfile: 192B                                                0.0s
 => [internal] load metadata for docker.io/library/python:3.11-slim                 1.1s
 => [internal] load .dockerignore                                                   0.0s
 => => transferring context: 2B                                                     0.0s
 => [1/4] FROM docker.io/library/python:3.11-slim@sha256:fba6f3b73795df99960f4  0.0s
 => [internal] load build context                                                   0.0s
 => => transferring context: 4.51kB                                                 0.0s
 => CACHED [2/4] WORKDIR /app                                                       0.0s
 => CACHED [3/4] RUN pip install flask==2.3                                         0.0s
 => [4/4] COPY . /app                                                               0.0s
 => exporting to image                                                              0.0s
 => => exporting layers                                                             0.0s
 => => writing image sha256:2946244ef4986fc8db3862b03497ff54044cf559ecbc202c86  0.0s
 => => naming to docker.io/library/sample-flask-app:v3                              0.0s
```

Step 6: Verify Image Creation

docker images

Step 7: Run Docker Container

docker run -d -p 5000:5000 sample-flask-app:v1

```
C:\Users\Lab805_19\Desktop\hi\flask-docker-app>docker run -d -p 5000:5000 sample-flas
k-app:v3
9e7cfc18b004a0063f6d38241975d34e22e91fa5ae092da8f6623a6c6b782ee7
docker: Error response from daemon: driver failed programming external connectivity o
n endpoint tender_lichterman (2a1906fac8ef492b97ff424614c3faa0034fe3cf609c886f5278e5c
2dbf9e760): Bind for 0.0.0.0:5000 failed: port is already allocated.
```

Step 8: Verify Running Container

docker ps

```
C:\Users\Lab805_19\Desktop\hi\flask-docker-app>docker ps
CONTAINER ID   IMAGE                 COMMAND                  CREATED         STATUS
        PORTS                    NAMES
f451e5f01f76   sample-flask-app:v3   "flask run --host=0.…"   7 minutes ago   Up 7 mi
nutes   0.0.0.0:5000->5000/tcp   sweet_turing
```

Step 9: Test Application in Browser

Open:

http://localhost:5000

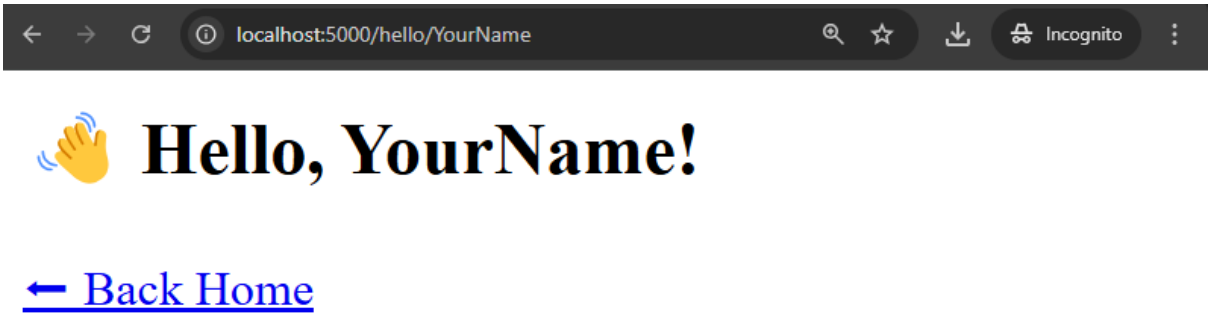
🚀 Welcome to My Advanced Docker Flask App

Select a feature below:

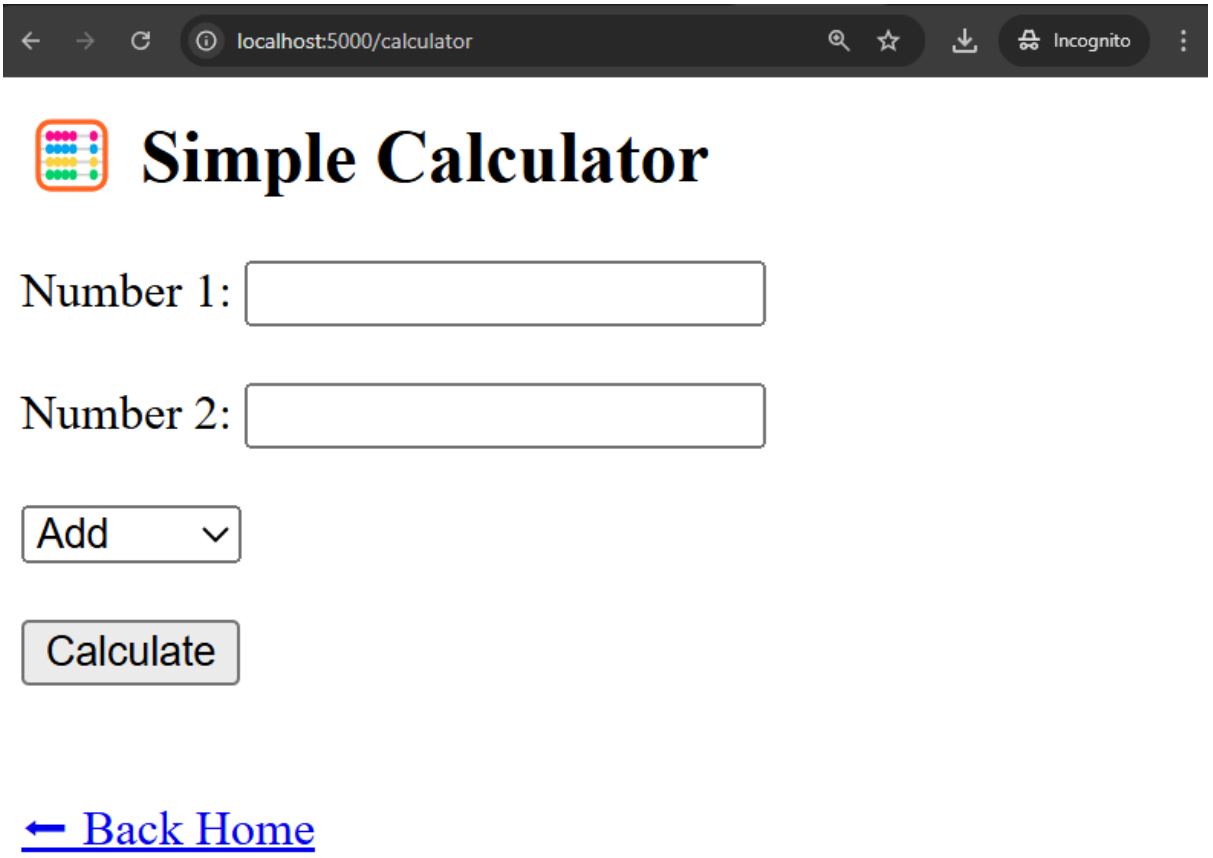
- Dynamic Hello
- Quick Add (5 + 10)
- Web Calculator
- Simple Form
- JSON API

Test additional routes:

http://localhost:5000/hello/YourName



http://localhost:5000/calculator



http://localhost:5000/api/data

localhost:5000/api/data

Pretty-print


```
{
  "features": [
    "Dynamic Routing",
    "Forms",
    "Calculator",
    "Dockerized"
  ],
  "message": "This is JSON data",
  "status": "success"
}
```

Output

The Flask application was successfully containerized using Docker.
The Docker image was built and the container was executed.
All routes including dynamic routing, form submission, calculator functionality, and JSON API were tested successfully.

Conclusion

In this experiment, we:

- Learned Dockerfile instructions
- Built a Docker image
- Created and ran a Docker container
- Implemented multiple Flask routes
- Tested the web application in a browser

The experiment demonstrates practical implementation of Docker with a Python Flask web application.